SERVRELESS RESUME HOSTING SYSTEM ON AWS

A SUMMER INTERNSHIP REPORT

Submitted by

JANARANJANI . M – 810023205097



.

DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING – (BIT CAMPUS)
ANNA UNIVERSITY ,
TIRUCHIRAPPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING – (BIT CAMPUS)
ANNA UNIVERSITY , TIRUCHIRAPPALLI – 620 024
DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Serverless Resume Hosting System On AWS"** is the Bonafide work of **JANARANJANI M (810023205097)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on __.

ABSTRACT

The **Serverless Resume Hosting System on AWS** is a cloud-based web application designed to provide a simple, scalable, reliable, and cost-effective platform for hosting and accessing resumes online. Traditional resume hosting methods often require users to maintain web servers, configure hosting environments, manage storage, and handle infrastructure-related tasks. These requirements can increase complexity and maintenance overhead, especially for students, job seekers, and professionals who need a simple platform to publish their resumes. This project addresses these challenges by utilizing **serverless cloud computing services provided by Amazon Web Services (AWS)**.

The proposed system uses a serverless architecture in which the underlying server infrastructure is managed by AWS. The resume content can be stored using **Amazon S3**, which provides secure and highly durable object storage. The application can use **AWS Lambda** to execute backend functions without requiring a continuously running server. **Amazon API Gateway** can be used to provide secure HTTP endpoints for communication between the client application and backend services. Through this architecture, users can access their resumes through a web interface without directly managing physical or virtual servers.

The system focuses on improving **availability, scalability, security, and cost efficiency**. Since server resources are provisioned and managed automatically by the cloud provider, the system can handle varying levels of traffic without requiring manual server configuration. The serverless approach also reduces operational responsibilities such as server maintenance, software installation, and infrastructure management. AWS Identity and Access Management (IAM) can be used to control access to cloud resources and apply appropriate permissions.

The main objective of this project is to demonstrate how serverless cloud technologies can be applied to develop and deploy a practical web-based application. The system provides an efficient solution for hosting resumes while demonstrating important cloud computing concepts such as **serverless architecture, object storage, API-based communication, automatic scalability, security, and pay-per-use resource utilization**. Overall, the Serverless Resume Hosting System on AWS offers a flexible and modern approach to online resume hosting and serves as a practical implementation of cloud-based application deployment.

CONTENTS

CHAPTER NO	TABLE OF CONTENTS	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP 1.2 OBJECTIVE OF THE FILE ENCRYPTION AND DECRYPTION USING AES ALGORITHM	05
2	COMPANY PROFILE	06
3	DAILY WORK REPORT	07-10
4	BLOCK DIAGRAM	10
5	SERVERLESS RESUME HOSTING SYSTEM 5.1. EXCITING SOLUTION 5.2 PROPOSED SOLUTION 5.3 CAPABILITIES	11
6	PROPOSED METHODS 6.1 AWS-BASED ACCESS CONTROL 6.2 RESUME STORAGE MODULE 6.3 SERVERLESS PROCESSING MODULE 6.4 API MANAGEMENT MODULE 6.5 STATIC WEBSITE HOSTING MODULE 6.6 RESUME ACCESS AND MONITORING	12-15
7	ALGORITHMS	15
8	CODE IMPLEMENTATION	16-26
9	SAMPLE OUTPUT	26
10	ANALYSIS	27-28
11	CONCLUSION	28
12	FUTURE WORK	29
13	REFERENCES	29

1. INTRODUCTION

Cloud computing is a modern technology that provides computing resources such as storage, processing power, networking, and application services through the internet. It enables users and organizations to use IT resources without the need to purchase and maintain physical infrastructure. Amazon Web Services (AWS) is one of the leading cloud platforms that provides a wide range of services for developing, deploying, and managing cloud-based applications. The **Serverless Resume Hosting System on AWS** is a cloud-based project developed to host and provide access to resumes through the internet. The system uses serverless technologies to reduce infrastructure management and provide a simple, scalable, reliable, and cost-effective hosting solution. Services such as **Amazon S3, AWS Lambda, and Amazon API Gateway** can be integrated to store resume files, execute application logic, and handle API communication. The project demonstrates important cloud computing concepts such as serverless architecture, cloud storage, scalability, security, availability, and efficient resource utilization.

1.1 Objective of the Internship

- To understand the fundamental concepts of Cloud Computing.
- To gain practical knowledge of Amazon Web Services (AWS).
- To learn about AWS services such as EC2, S3, IAM, Lambda, and API Gateway.
- To understand serverless computing and cloud-based application deployment.
- To gain hands-on experience in cloud storage and resource management.
- To understand security, scalability, availability, and cost optimization in cloud environments.

1.2 Objective of the serverless resume hosting system on aws

- To develop a serverless platform for hosting resumes online.
- To store resume files and static content using Amazon S3.
- To use AWS Lambda for executing backend application logic without managing servers.
- To use Amazon API Gateway for handling API requests and communication.
- To reduce the need for server maintenance and infrastructure management.
- To provide secure and reliable access to resume content.
- To demonstrate the scalability and flexibility of serverless architecture.
- To develop a cost-effective and easily maintainable resume hosting solution.

2.COMPANY PROFILE

Fantasy Solution:

Trichy is a leading IT solutions and service provider offering software development, web designing, web hosting, and IT training services.

The organization specializes in real-time project development, application development & maintenance, ERP implementation & consulting, testing, performance engineering, and software localization.

Fantasy Solution has contributed to the formulation of several national IT standards and continues to deliver technology solutions in fields such as automotive electronics, smart devices, digital home products, and enterprise applications.

Address:
Fantasy Solution,
No 16, Samnath Plazza,
Third Floor, Near
BG Naidu Sweets,
Melapudur, Trichy
– 1.
Contact: 9043095535
Landline: 0431-4971630 Website: www.fantasysolution.in

Mission:

Fantasy Solution is committed to providing high quality software development services, secure access to information, improved communication between businesses, value-formoney solutions to customers, and meaningful growth opportunities for its employees.

Vision:

To be a leading software development and training organization by delivering innovative, high-quality, and industry-oriented technology solutions. The company aims to empower students and organizations through focused project guidance, practical learning, and professional software development services that enhance technical skills and support career growth. It strives to bridge the gap between academic learning and industry requirements by providing real-time project experience and continuous technological innovation.

3. DAILY WORK REPORT:

Domain: CLOUD COMPUTING
Project : Serverless Resume Hosting System On AWS
Company: Fantasy IT Solutions
DAILY WORK REPORT

Day 1
Introduction to the internship, project requirements, and basic concepts of cloud computing. Learned how cloud computing provides computing resources through the internet and discussed the overall objectives of the internship project.

Day 2
Studied the fundamentals of cloud computing, including its characteristics, benefits, and service models such as IaaS, PaaS, and SaaS. Understood how cloud services reduce infrastructure requirements and support flexible resource utilization.

Day 3
Introduction to Amazon Web Services (AWS) and explored the AWS Management Console. Studied the major AWS services used for computing, storage, networking, security, and application development.

Day 4
Studied AWS global infrastructure, including Regions, Availability Zones, and Edge Locations. Learned how AWS infrastructure helps provide high availability, reliability, and efficient access to cloud resources.

Day 5
Learned AWS Identity and Access Management (IAM), including users, roles, policies, and permissions. Understood the importance of authentication, authorization, and secure access to AWS resources.

Day 6
Studied Amazon EC2 and learned the basic concepts of virtual machines, instances, machine images, storage, and instance management. Understood how EC2 provides scalable computing capacity in the cloud.

Day 7
Practiced launching and configuring an EC2 instance. Learned about instance types, key pairs, security groups, and basic network security configurations required for accessing an EC2 instance.

Day 8
Introduction to Amazon S3 and studied buckets, objects, storage classes, and storage concepts. Learned how S3 provides scalable object storage for files and application data.

Day 9

Created an S3 bucket and practiced uploading, downloading, viewing, and managing files. Studied the basic organization of objects within S3 buckets.

Day 10

Studied S3 security, bucket policies, permissions, and static website hosting concepts. Learned how access to stored objects can be controlled using appropriate permissions and policies.

Day 11

Introduction to serverless computing and studied its architecture, advantages, limitations, and applications. Understood how serverless computing reduces the need for direct server management.

Day 12

Studied AWS Lambda, including functions, triggers, events, runtime environments, and execution. Learned how Lambda executes application code in response to events without requiring dedicated servers.

Day 13

Created and tested a basic AWS Lambda function using sample events. Practiced configuring the function, providing input events, executing the function, and observing the output.

Day 14

Studied Amazon API Gateway and learned how APIs connect applications with backend services. Understood API endpoints, HTTP methods, requests, responses, and integration with Lambda.

Day 15

Created and tested a basic API Gateway endpoint integrated with an AWS Lambda function. Verified that requests sent through the API could successfully invoke the Lambda function and return responses.

Day 16

Analyzed the requirements of the Serverless Resume Hosting System and identified the required AWS services. Studied how S3, Lambda, API Gateway, and IAM could work together to implement the proposed system.

Day 17

Designed the basic architecture of the Serverless Resume Hosting System using AWS services. Defined the flow between the user interface, API Gateway, Lambda functions, and S3 storage.

Day 18

Created the project S3 bucket and configured storage for resume files and web resources. Organized the required files and verified that the objects were stored correctly.

Day 19

Developed the basic frontend structure of the resume hosting system using HTML and CSS. Created the required page layout and designed the interface for displaying resume information.

Day 20

Integrated the frontend resources with Amazon S3 and tested the hosted content. Verified that the web resources could be accessed correctly through the configured hosting environment.

Day 21

Developed the required AWS Lambda function for handling backend operations. Implemented the required application logic and tested the function with different input conditions.

Day 22

Connected the Lambda function with API Gateway and configured the required API endpoint. Tested the API configuration and verified successful communication between the API and Lambda function.

Day 23

Tested communication between the frontend, API Gateway, and Lambda function. Verified request and response flow and checked whether the system produced the expected results.

Day 24

Configured appropriate IAM permissions and reviewed the security settings of AWS resources. Ensured that required services had suitable permissions while avoiding unnecessary access.

Day 25

Tested resume uploading, storage, and retrieval functionality. Verified that resume files were correctly stored in S3 and could be accessed through the application.

Day 26

Identified and resolved configuration, API, permission, and integration-related issues. Re-tested the affected components after making the required corrections.

Day 27

Performed functional testing of the complete Serverless Resume Hosting System. Tested the major system functions and verified the interaction between the frontend and AWS backend services.

Day 28

Tested system accessibility and reviewed the reliability, scalability, and AWS resource usage.

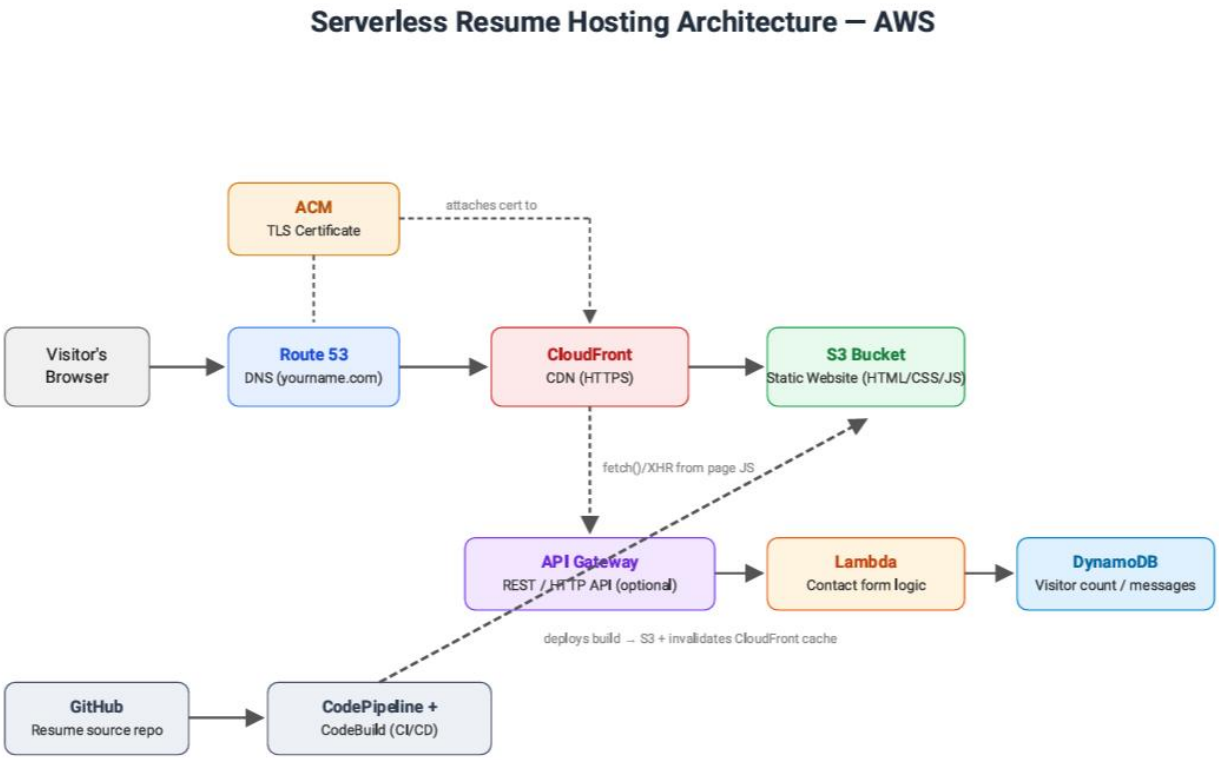
Day 29

Finalized the project implementation and prepared project screenshots and documentation. Reviewed the system architecture, AWS services used, implementation steps, and testing results.

Day 30

Completed the project report and reviewed the complete Serverless Resume Hosting System.

4.BLOCK DIAGRAM:



5. SERVERLESS RESUME HOSTING SYSTEM ON AWS

The Serverless Resume Hosting System on AWS is a cloud-based solution designed to host and provide online access to resume documents without requiring a dedicated traditional web server. The system uses AWS cloud services to provide reliable storage, application processing, and secure access to resume content. A serverless architecture reduces the need for infrastructure management because the underlying servers are managed by AWS. The system can use Amazon S3 for storing resume files and static web resources, AWS Lambda for executing backend application logic, and Amazon API Gateway for handling API requests. AWS IAM can be used to control access to the required cloud resources. The proposed system demonstrates the practical

use of serverless computing for developing a simple, scalable, and cost-effective resume hosting platform.

5.1 EXISTING SOLUTION

Traditional resume hosting generally depends on personal computers, local storage, shared hosting platforms, or conventional web servers. In a server-based hosting environment, the user or administrator may need to manage the server, operating system, storage, software configuration, security settings, and maintenance activities.

The major limitations of the existing approach include:

- Requires a **server or hosting environment** to publish the resume.
- Infrastructure may remain active even when there are very few users.
- Scaling resources manually can be difficult when the number of requests increases.
- Server maintenance can increase operational effort.
- Storage and hosting configuration may require additional management.
- Security settings must be properly configured and maintained.
- Traditional hosting may not efficiently utilize resources for applications with low or irregular traffic.

5.2 PROPOSED SOLUTION

The proposed Serverless Resume Hosting System on AWS addresses the limitations of traditional hosting by using managed cloud services and a serverless architecture. Instead of maintaining a dedicated server, the system uses AWS services to store files, execute application logic, and handle requests.

The resume and required static resources can be stored in Amazon S3. When backend processing is required, AWS Lambda executes the required functions without the need to maintain a continuously running server. Amazon API Gateway provides an interface for receiving application requests and forwarding them to the appropriate Lambda function. AWS IAM manages authentication and authorization through roles and policies.

The proposed solution provides:

- Serverless application deployment.
- Cloud-based resume storage using Amazon S3.
- Backend processing using AWS Lambda.
- API communication using Amazon API Gateway.
- Controlled access using AWS IAM.
- Reduced server maintenance requirements.
- Automatic resource scaling based on demand.
- Reliable and easily accessible resume hosting.

- Flexible and cost-efficient cloud resource utilization.

5.3 CAPABILITIES

- **Online Resume Hosting** – Hosts resumes and makes them accessible through the internet.
- **Cloud Storage** – Stores resume files securely using Amazon S3.
- **Serverless Processing** – Executes backend functions using AWS Lambda.
- **API Integration** – Handles application requests using Amazon API Gateway.
- **Secure Access Control** – Manages permissions using AWS IAM.
- **Scalability** – Supports changing user requests without manual server scaling.
- **High Availability** – Provides reliable access to hosted resume content.
- **Cost Efficiency** – Reduces infrastructure and server maintenance costs.
- **Easy Maintenance** – Eliminates the need to manage physical or virtual servers.
- **Remote Accessibility** – Allows resumes to be accessed from anywhere with an internet connection.
- **Fast Deployment** – Enables quick deployment using managed AWS services.
- **Resource Management** – Provides centralized management of cloud-based resources.

6. PROPOSED METHOD

Serverless Resume Hosting System on AWS

The proposed **Serverless Resume Hosting System on AWS** is designed to provide a reliable, scalable, and cost-effective platform for hosting resumes online. The system uses AWS serverless and cloud storage services to eliminate the need for maintaining a dedicated web server. Resume files and website resources can be stored using **Amazon S3**, while backend operations can be handled using **AWS Lambda**. **Amazon API Gateway** can be used to provide API communication between the frontend application and backend services.

The system provides an environment where users can upload, store, manage, and access resume files through a web-based interface. AWS **IAM** can be used to control access to cloud resources and provide appropriate permissions. The serverless architecture reduces infrastructure management and allows the application to scale according to demand.

6.1 AWS-Based Access Control and Security

The proposed system uses AWS security mechanisms to control access to project resources. **AWS Identity and Access Management (IAM)** provides users, roles, and policies for managing permissions.

Features**:**

- Provides controlled access to AWS resources.
- Uses IAM roles and policies for authorization.
- Restricts access to required S3, Lambda, and API Gateway resources.
- Prevents unauthorized modification of project resources.
- Supports secure management of cloud resources.

Roles:

- **Administrator:** Manages AWS resources, permissions, and system configuration.
- **Developer:** Develops and maintains Lambda functions and application components.
- **User:** Accesses and views the hosted resume through the application.

6.2 Resume Storage Module

The Resume Storage Module is responsible for storing resume files and required website resources in **Amazon S3**.

Features:

- Allows resume files to be uploaded to cloud storage.
- Stores resume documents as objects in an S3 bucket.
- Provides scalable storage for resume files.
- Allows authorized users to retrieve stored resumes.
- Maintains organized storage of project files.
- Provides reliable access to stored content.

Process**:**

- User selects and uploads a resume.
- The application receives the resume file.
- The file is stored as an object in the configured S3 bucket.
- S3 manages the storage of the uploaded file.
- The stored resume can be accessed through the application.

6.3 Serverless Processing Module

The Serverless Processing Module uses **AWS Lambda** to execute backend operations without requiring a continuously running server.

Features**:**

- Executes application logic without server management.
- Processes requests received from the application.
- Performs required resume-related operations.
- Automatically provides computing resources when functions are invoked.
- Reduces infrastructure management requirements.

Process**:**

- User performs an operation through the application.
- The request is sent to the backend through the API.
- API Gateway invokes the required Lambda function.
- Lambda processes the request.
- The function returns the required response to the application.

6.4 API Management Module

The API Management Module uses **Amazon API Gateway** to provide communication between the frontend application and backend Lambda functions.

Features:

- Provides API endpoints for application requests.

- Connects the frontend with AWS Lambda.
- Supports HTTP-based communication.
- Processes client requests and responses.
- Provides a managed interface for backend services.

Process**:**

- User interacts with the resume hosting application.
- The application sends an API request.
- API Gateway receives and processes the request.
- API Gateway invokes the appropriate Lambda function.

6.5 Static Website Hosting Module

The Static Website Hosting Module provides the frontend interface required to display resume information. Website files such as HTML, CSS, and JavaScript can be stored in **Amazon S3**.

Features:

- Hosts static website resources using cloud storage.
- Provides a web interface for displaying resume information.
- Allows users to access the resume through the internet.
- Reduces the need for a traditional web server.
- Supports easy deployment and updating of website files.

6.6 Resume Access and Monitoring

The system manages resume access and monitors important application activities to improve reliability and security.

Features**:**

- Provides controlled access to hosted resume files.
- Tracks important application operations.
- Helps identify access and configuration issues.
- Supports monitoring of AWS resources.
- Improves overall system reliability.
- Helps maintain proper management of cloud resources.

Advantages of Proposed System

- Provides **serverless resume hosting** using AWS.
- Eliminates the need to maintain dedicated servers.
- Provides scalable cloud-based storage using Amazon S3.
- Reduces infrastructure management requirements.
- Provides secure resource access using AWS IAM.
- Supports backend processing using AWS Lambda.
- Provides API communication through Amazon API Gateway.
- Offers reliable access to hosted resume content.
- Supports flexible and efficient resource utilization.
- Provides a cost-effective approach for resume hosting.

- Allows resumes to be accessed from anywhere through the internet.
- Demonstrates practical implementation of serverless cloud computing.

Disadvantages of Proposed Solution

Serverless Resume Hosting System on AWS

1. AWS Service Dependency
   - The system depends on AWS services for hosting, storage, and backend processing.
   - Changes in AWS service availability or configuration can affect the application.
   - Moving the application to another cloud provider may require modifications.
2. Internet Dependency
   - The resume requires an internet connection for online access.
   - Users with poor or unavailable internet connectivity may experience difficulty accessing the hosted resume.
   - Network issues can affect application accessibility.
3. Serverless Limitations
   - Serverless functions have limitations related to execution time and available resources.
   - Complex or resource-intensive operations may require additional architectural planning.
   - Developers need to understand the configuration of multiple managed AWS services.
4. Cost Management
   - Although serverless architecture can be cost-efficient, improper configuration or unexpected usage can result in AWS charges.
   - Cloud resources must be monitored regularly to avoid unnecessary costs.
   - Additional AWS services may increase the overall operational cost.
5. Configuration and Security Complexity
   - Incorrect IAM permissions or S3 configurations can create security risks.
   - Proper access policies must be configured to protect resume files and cloud resources.
   - Managing multiple AWS services requires appropriate technical knowledge.

7. ALGORITHM

7.1 Algorithm for Resume Upload and Storage Using Amazon S3

Process 1: Resume Upload

1. The user selects the resume file through the web application.
2. The system receives the selected resume through the upload interface.
3. The uploaded file is validated for file type and size.
4. The system prepares the resume file for cloud storage.
5. The resume file is sent to the configured AWS storage service.

Process 2: Resume Storage in Amazon S3

1. The system connects to the configured Amazon S3 bucket.
2. The resume file is uploaded as an object into the S3 bucket.
3. The system assigns the required object name or file path.
4. S3 stores the resume file securely in cloud storage.

5. The system verifies that the upload operation is completed successfully.

Process 3: Resume Metadata Management

1. The system identifies the uploaded resume details.
2. File information such as file name, file type, and file size is processed.
3. The required metadata is associated with the stored resume.
4. The system maintains the information required for resume retrieval.
5. The storage operation is completed successfully.

Process 4: Upload Confirmation

1. The system checks the status of the S3 upload operation.
2. If the upload is successful, the system generates a success response.
3. The application displays the upload confirmation to the user.
4. If the upload fails, the system displays an appropriate error message.
5. The resume remains available in S3 after successful storage.

7.2 Algorithm for Resume Retrieval Using AWS Lambda and API Gateway

Process 1: Resume Access Request

1. The user opens the resume hosting application.
2. The user requests to view or access the hosted resume.
3. The application generates a request for the required resume.
4. The request is sent to the configured API endpoint.
5. Amazon API Gateway receives the request.

Process 2: API Request Processing

1. API Gateway receives the user's request.
2. The request is forwarded to the appropriate AWS Lambda function.
3. Lambda processes the received request.
4. The system verifies the required resource and access permissions.
5. Lambda prepares the response for the requested resume.

Process 3: Resume Retrieval from Amazon S3

1. The Lambda function identifies the required resume object.
2. The function requests the resume from the configured S3 bucket.
3. Amazon S3 retrieves the stored resume object.
4. The retrieved resume data is processed by the application.
5. Lambda prepares the required response.

Process 4: Resume Display and Access

1. The Lambda function returns the response through API Gateway.
2. API Gateway sends the response to the client application.
3. The application receives the resume information or resource location.
4. The resume is displayed or made available to the authorized user.
5. The system completes the resume retrieval process successfully.

7.3 Algorithm for Serverless Resume Hosting

Process 1: User Request

1. The user accesses the resume hosting website.
2. The user selects the required resume operation.
3. The frontend generates an appropriate request.
4. The request is sent to the required AWS service.

Process 2: Serverless Request Processing

1. API Gateway receives the application request.
2. API Gateway invokes the appropriate AWS Lambda function.
3. Lambda executes the required backend logic.
4. Lambda communicates with Amazon S3 when storage access is required.
5. The result is returned to the frontend through API Gateway.

Process 3: Response Handling

1. The frontend receives the response from the backend.
2. The system checks whether the requested operation was successful.
3. The requested resume information or file is displayed to the user.
4. If an error occurs, an appropriate error message is displayed.
5. The process is completed after the user receives the requested result.

8.CODE IMPLEMENTATION:

```
// RESUME
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Janaranjani M - Resume</title>
  <style>
    * {
      box-sizing: border-box;
    }
    body {
      margin: 0;
      padding: 0;
      background: white;
      color: #111;
      font-family: "Times New Roman", serif
```

```
    font-size: 11px;
}
   .resume {
   width: 210mm;
   min-height: 297mm;
   margin: auto;
   padding: 12mm 14mm;
   background: white;
}
/* Header */
.header {
   text-align: center;
}
.header h1 {
   margin: 0;
   font-size: 25px;
   letter-spacing: 1px;
}
.designation {
   margin-top: 3px;
   font-size: 13px;
}
.contact {
   margin-top: 8px;
   font-size: 10px;
}
.contact span {
   margin: 0 8px;
}
/* Sections */
.section {
   margin-top: 13px;
}
.section-title {
   display: flex;
   justify-content: space-between;
   align-items: center;
   font-size: 13px;
   font-weight: bold;
```

```
    border-bottom: 1px solid #777;
    padding-bottom: 3px;
    margin-bottom: 7px;
}
/* About */
.about {
    text-align: justify;
    line-height: 1.35;
    margin: 0;
}
/* Education */
.education {
    text-align: center;
}
.education .degree {
    font-weight: bold;
    font-size: 12px;
}
.education .college {
    margin-top: 3px;
}
.education-details {
    display: flex;
    justify-content: space-between;
    margin-top: 4px;
}
/* Internship */
.entry-title {
    text-align: center;
    font-weight: bold;
    font-size: 12px;
}
.entry-info {
    display: flex;
    justify-content: space-between;
    margin-top: 4px;
}
ul {
    margin-top: 7px;
    padding-left: 17px
```

```
li {
    margin-bottom: 6px;
    line-height: 1.35;
}

/* Certificate */

.certificate-title {
    text-align: center;
    font-weight: bold;
    font-size: 12px;
}
/* Project */
.project-title {
    text-align: center;
    font-weight: bold;
    font-size: 12px;
}
.project ul {
    margin-top: 7px;
}

/* Skills */
.skills-table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 4px;
}
.skills-table td {
    padding: 2px 0;
    vertical-align: top;
}
.skills-table td:first-child {
    width: 25%;
    padding-left: 40px;
    font-weight: bold;
}
.skills-table td:nth-child(2) {
    width: 75%;
```

```
    }
    /* Print */
    @media print {
      body {
        background: white;
      }
      .resume {
        width: 210mm;
        min-height: 297mm;
        margin: 0;
        padding: 12mm 14mm;
      }
    }
  </style>
</head>

<body>

<div class="resume">

  <!-- HEADER -->
  <div class="header">

    <h1>JANARANJANI M</h1>

    <div class="designation">
      Aspiring Cloud Engineer
    </div>

    <div class="contact">
      <span>☎ +91-8122056069</span>
      <span>✉ janaranjaninmchamy@gmail.com</span>
      <span>◉ github.com/janaahub</span>
      <span>in Janaranjani-m</span>
    </div>

  </div>


  <!-- ABOUT -->
```

```
<div class="section">

  <div class="section-title">
    <span>ABOUT</span>
    <span>ME</span>
  </div>

  <p class="about">
    Motivated B.Tech Information Technology student with a keen interest in
    Cloud Computing and Amazon Web Services (AWS). Possesses foundational
    knowledge of Java programming and core computer science concepts, with a
    strong willingness to learn, solve real-world problems, and build practical
    cloud-based solutions.
  </p>

</div>
<!-- EDUCATION -->
<div class="section">

  <div class="section-title">
    <span>EDUCATION</span>
  </div>

  <div class="education">

    <div class="degree">
      Bachelor of Technology (B.Tech) – Information Technology
    </div>

    <div class="college">
      University College of Engineering BIT Campus
    </div>

    <div class="education-details">
      <span>CGPA: 7.93 (Up to VI Semester)</span>
      <span>2023–2027</span>
    </div>

  </div>
```

```
<!-- INTERNSHIP -->
<div class="section">

  <div class="section-title">
    <span>INTERNSHIP</span>
  </div>

  <div class="entry-title">
    Cloud Computing Intern
  </div>

  <div class="entry-info">
    <span>Fantasy Solution</span>
    <span>June 15, 2026 – July 15, 2026</span>
  </div>

  <ul>
    <li>
      Gained hands-on experience with AWS Cloud services including
      EC2, S3, IAM, and VPC while learning cloud infrastructure,
      virtualization, and storage concepts.
    </li>

    <li>
      Implemented IAM-based access control and gained hands-on
      experience with cloud resource management and application deployment.
    </li>
  </ul>

</div>


<!-- CERTIFICATE -->
<div class="section">

  <div class="section-title">
    <span>CERTIFICATE</span>
  </div>

  <div class="certificate-title">
```

```
    Docker Containerization Intern
  </div>

  <ul>
    <li>
      Learned Docker and containerization concepts by creating and
      managing Docker containers and custom images using Dockerfiles.
    </li>

    <li>
      Deployed multi-container applications using Docker Compose and
      gained hands-on experience with container networking, volumes,
      and image management.
    </li>
  </ul>

</div>


<!-- PROJECT -->
<div class="section project">

  <div class="section-title">
    <span>PROJECT</span>
  </div>

  <div class="project-title">
    Serverless Resume Hosting System on AWS
  </div>

  <ul>
    <li>
      Built a serverless resume hosting system using AWS S3,
      CloudFront, API Gateway, Lambda, and DynamoDB for secure
      and scalable web hosting.
    </li>

    <li>
      Configured static website hosting, integrated a visitor
      counter using AWS Lambda and DynamoDB, and managed access
```

```
            using AWS IAM.
          </li>
        </ul>

    </div>
//  AWS LAMBDA FUNCTION
exports.handler = async (event) => {
    const response = {
        statusCode: 200,
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            message: "Resume Hosting System is working successfully",
            status: "Success"
        })
    };

    return response;
};

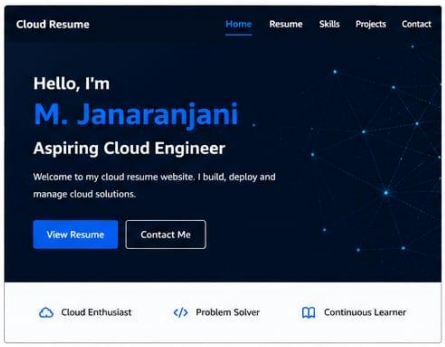
// LAMBDA FUNCTION FOR RESUME RETRIEVAL FROM S3
const AWS = require("aws-sdk");
const s3 = new AWS.S3();
exports.handler = async (event) => {
    const params = {
        Bucket: "your-resume-bucket",
        Key: "resume.pdf"
    };
    try {
        const data = await s3.getObject(params).promise();
        return {
            statusCode: 200,
            headers: {
                "Content-Type": "application/pdf"
            },
            body: data.Body.toString("base64"),
            isBase64Encoded: true
        };
     } catch (error) {
```

```
        return {
            statusCode: 500,
            body: JSON.stringify({
                message: "Unable to retrieve resume",
                error: error.message
            })
        };
    }
};

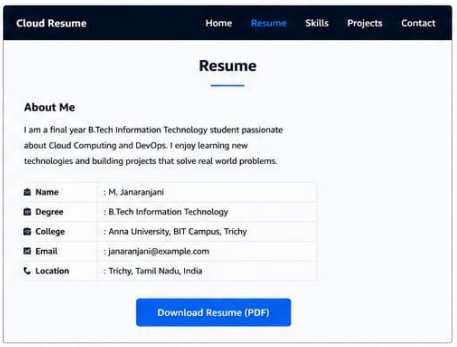
// FRONTEND API REQUEST
async function getResume() {
    const response = await fetch(
        "YOUR_API_GATEWAY_URL"
    );
    const data = await response.json();
    console.log(data);
}
```

SAMPLE OUTPUT

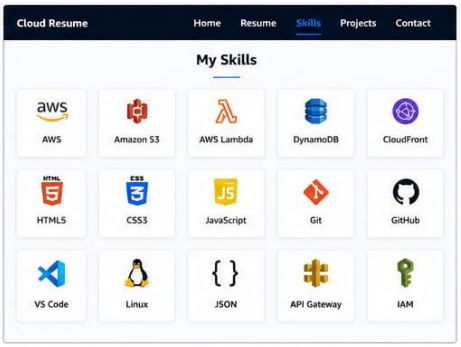
1. Home Page



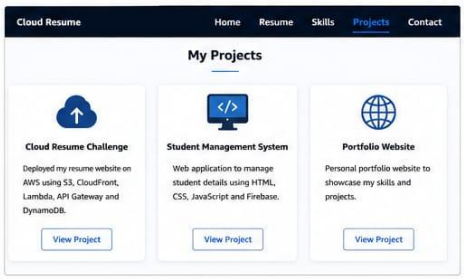
2. Resume Page



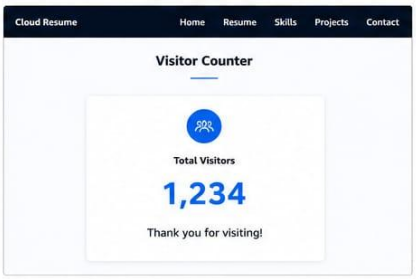
3. Skills Section



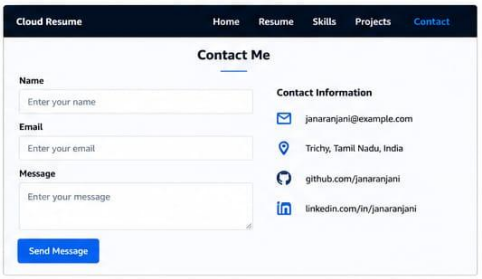
4. Projects Section



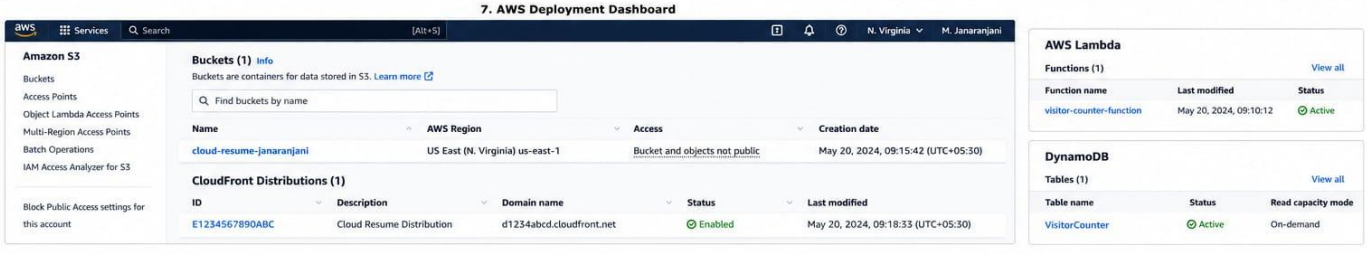
5. Visitor Counter



6. Contact Page



7. AWS Deployment Dashboard



10. ANALYSIS

1. AWS Resource and Configuration Analysis
   - The system uses **Amazon S3** for storing resume files and static website resources.
   - **AWS Lambda** is used to execute backend operations without maintaining a dedicated server.
   - **Amazon API Gateway** handles communication between the frontend application and Lambda functions.
   - **AWS IAM** manages access permissions for AWS resources.
   - Proper configuration of AWS resources is required to ensure secure and reliable application operation.
2. Resume Upload and Storage Analysis
   - The user selects and uploads the resume through the web application.
   - The uploaded resume is transferred to the configured **Amazon S3 bucket**.
   - S3 stores the resume as an object in cloud storage.
   - File details such as file name, file type, and size can be managed by the application.
   - Cloud-based storage provides reliable access to resume files.
3. Serverless Processing Analysis
   - **AWS Lambda** processes backend requests without requiring a continuously running server.
   - Lambda functions are invoked when specific application events or API requests occur.
   - The serverless approach reduces infrastructure management requirements.
   - Computing resources are automatically provided when the function is executed.
   - This approach is suitable for applications with variable or irregular workloads.
4. API Communication Analysis
   - **Amazon API Gateway** provides API endpoints for communication with the application.
   - User requests are received by API Gateway and forwarded to the appropriate Lambda function.
   - Lambda processes the request and generates the required response.
   - The response is returned to the frontend through API Gateway.
   - This provides a structured communication path between the frontend and backend.
5. Security Analysis
   - **AWS IAM** is used to control access to AWS resources.
   - IAM roles and policies can restrict Lambda and other services to only the permissions they require.
   - S3 access can be controlled using appropriate bucket and object permissions.
   - Proper security configuration helps prevent unauthorized access to resume files.
   - Least-privilege access should be followed when configuring AWS permissions.
6. Scalability Analysis

- The serverless architecture supports automatic scaling based on application demand.
- AWS Lambda can execute multiple function instances when request volume increases.
- Amazon S3 provides scalable storage for resume files.
- The system does not require manual server provisioning for normal changes in demand.
- This makes the solution suitable for applications with varying numbers of users.

7. Availability and Reliability Analysis
   - AWS managed services provide a reliable environment for hosting application components.
   - Resume files stored in Amazon S3 can be accessed through the configured application.
   - Separating storage, API handling, and backend processing reduces dependency on a single traditional server.
   - The system can continue to provide services without requiring manual server maintenance.
   - Regular monitoring and proper configuration improve overall system reliability.
8. Performance and Cost Analysis
   - Serverless execution reduces the need for continuously running computing infrastructure.
   - AWS Lambda executes backend code when required, which can improve resource utilization.
   - Amazon S3 provides efficient storage and retrieval of resume files.
   - API Gateway provides managed API communication between application components.
   - The serverless model can be cost-effective for applications with low or variable traffic, although AWS resource usage must be monitored to avoid unexpected charges.

11.**CONCLUSION:**

The 30-day internship at Fantasy Solution offered a thorough and structured learning experience, covering core Java fundamentals, advanced AES algorithm concepts, Hibernate, and security concepts.

Applying these concepts to the **Serverless Resume Hosting System on AWS** made the learning practical and outcome-driven. The project provided hands-on experience in deploying and managing a serverless web application using AWS services, while gaining practical knowledge of cloud computing, storage, hosting, and security.

This experience strengthened the understanding of **Java development, AWS cloud services, serverless architecture, and application deployment**, and has better prepared the intern for future software development and cloud computing roles.

12. FUTURE WORKS

- Integration of **AES-256 with RSA** for secure encryption key exchange (Hybrid Encryption).
- Development of a **cloud-based encryption system** for secure file storage and sharing.
- Implementation of **Two-Factor Authentication (2FA)** using OTP or authenticator applications for enhanced security.
- Addition of **biometric authentication** (fingerprint or facial recognition) before file encryption and decryption.
- Support for **multiple encryption algorithms** such as RSA, Blowfish, ChaCha20, and ECC, allowing users to choose the preferred encryption method.
- Development of **desktop, web, and mobile applications** to enable secure file access across different platforms.
- Integration of **digital signatures** to verify file authenticity and ensure non-repudiation.
- Automatic **secure key management** using encrypted key vaults or Hardware Security Modules (HSMs).
- Support for **batch encryption and decryption** to process multiple files simultaneously.
- Implementation of **secure file sharing** with password-protected encrypted links and expiry dates.
- Addition of **role-based access control (RBAC)** to restrict file access based on user roles and permissions.
- Integration with **cloud platforms** such as Google Drive, Dropbox, and Microsoft OneDrive for encrypted file synchronization.
- Implementation of **real-time audit logs** to monitor encryption, decryption, and file access activities.
- Support for **AI-based anomaly detection** to identify suspicious file access and potential cyber threats.
- Addition of **automatic backup and recovery** features for encrypted files.

13.REFERENCES

1. National Institute of Standards and Technology (NIST). FIPS PUB 197: Advanced Encryption Standard (AES). U.S. Department of Commerce, Gaithersburg, MD, USA, November 2001.
2. Dworkin, M. NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation – Galois/Counter Mode (GCM) and GMAC. National Institute of Standards and Technology, November 2007.
3. McGrew, D., RFC 5116: An Interface and Algorithms for Authenticated Encryption. Internet Engineering Task Force (IETF), January 2008.